

TaskFlow

チーム開発ドキュメント

ファイル構成・担当分類リファレンス

概要

このドキュメントは TaskFlow（課題管理アプリ）の開発に携わるチームメンバー向けに、ファイル構成と担当範囲を整理したリファレンスです。各担当が編集すべきファイルと、そのファイルが持つ関数・役割を一覧化しています。

ファイル構成

ファイル	役割
index.html	担当1 HTML構造・全画面の骨格
sw.js	担当2 Service Worker 本体
css/base.css	担当1 デザイントークン・レイアウト
css/components.css	担当1 共通UIコンポーネント
css/views.css	担当1 各画面固有スタイル
js/storage.js	担当3 localStorage 読み書き
js/task.js	担当2・3 タスクCRUD・バリデーション
js/label.js	担当4 ラベルCRUD・バリデーション
js/filter.js	担当4 絞り込みロジック
js/notification.js	担当2 SW登録・通知スケジュール同期
js/ui.js	担当1 DOM描画・画面遷移
js/main.js	全員 初期化・イベント結線

担当早見表

担当	領域	主なファイル
担当1	UI・見た目・画面遷移	index.html / css/* / js/ui.js

担当	領域	主なファイル
担当2	タスク更新・通知機能	sw.js / js/notification.js / js/task.js (更新系)
担当3	タスク登録・削除	js/task.js (登録・削除系) / js/storage.js
担当4	フィルタリング・ラベル登録	js/filter.js / js/label.js

担当1: UI(見た目)・画面遷移

css/base.css

アプリ全体の土台となるスタイルを定義するファイル。ここを変更すると見た目の基礎部分がすべて変わるため、変更時はチーム全員への共有が必要。

ファイル	役割
CSS変数 (:root)	カラー・フォント・角丸・影などのデザイントークンを一元管理。色を変えるときはここだけ編集すれば全体に反映される
body / .sidebar / .main	サイドバー (220px固定) + メインエリアの2カラムレイアウト
サイドバーナビ	.nav-item のホバー・アクティブ状態のスタイル
画面切り替え	.view / .view.active による表示・非表示制御

css/components.css

画面をまたいで使い回す部品（コンポーネント）のスタイルをまとめたファイル。新しいボタンや入力欄を追加する場合はここに追記する。

ファイル	役割
ボタン類	.btn-primary / .btn-secondary / .btn-danger / .btn-danger-outline / .btn-close / .btn-back
フォーム入力欄	input / select / textarea の共通スタイル。フォーカス時に青紫のボーダーが付く
種類トグル	.type-toggle / .type-btn — 課題（青紫）・テスト（琥珀）の2択ボタン
ステータス選択	.status-select / .status-btn — 未着手・進行中・完了の3択
バッジ類	.status-badge (未着手・進行中・完了) / .card-type-badge (課題・テスト)
ラベルチップ	.label-chip — ラベル管理画面のタグ表示
モーダル	.modal-overlay / .modal — 削除確認ダイアログ
トースト	.toast — 画面右下に表示される一時的な通知メッセージ

css/views.css

各画面に固有のスタイルを定義。担当1が各画面のレイアウトを調整する際の主な編集対象。

ファイル	役割
タブ(.tab-group)	ステータスフィルタ（すべて・未着手・進行中・完了）のタブUI

ファイル	役割
ラベルフィルタ	<code>.label-dropdown</code> — ラベル絞り込みのドロップダウンメニュー
タスクカードグリッド	<code>.task-grid</code> (auto-fill グリッド) / <code>.task-card</code> (カードデザイン)
<code>.urgency-bar</code>	【シグネチャ要素】カード上部の残り時間バー。通常=青紫、3日以内=琥珀、期限切れ=赤に変化
詳細画面	<code>.detail-card</code> / <code>.deadline-display</code> / <code>.status-change-row</code>
ラベル管理画面	<code>.label-input-row</code> / <code>.label-list-section</code>

js/ui.js

DOMの描画と画面遷移のロジックをすべて担当するモジュール。データを受け取ってHTMLに変換する役割のみを持ち、データの変更・保存は行わない。他モジュールから「描いてほしいデータ」を受け取り、画面に反映するだけにとどめる。

関数	役割
<code>showView(viewId)</code>	画面切り替え。 <code>viewId</code> は <code>list</code> / <code>add-task</code> / <code>detail</code> / <code>edit</code> / <code>add-label</code> の5種類
<code>renderTaskList(tasks, onCardClick)</code>	タスクカード一覧を描画。 <code>urgency-bar</code> の幅・色もここで計算
<code>renderDetail(task, onStatusChange)</code>	詳細画面のHTMLを生成。ステータス変更ボタンのイベントも設定
<code>resetAddForm(labels)</code>	登録フォームを初期状態（空欄・課題選択）にリセット
<code>populateEditForm(task, labels)</code>	編集フォームに既存タスクの値をセット
<code>getAddFormValues()</code>	登録フォームの入力値を収集してオブジェクトで返す
<code>getEditFormValues()</code>	編集フォームの入力値を収集してオブジェクトで返す
<code>renderLabelChips(labels, onDelete)</code>	ラベル管理画面のチップ一覧を描画
<code>renderLabelFilterList(labels, selected, onToggle)</code>	ラベルフィルタのチェックリストを描画
<code>syncLabelSelects(labels, selected)</code>	登録・編集フォームのラベル選択肢を最新状態に更新
<code>showToast(msg, type)</code>	画面右下にトーストメッセージを2.8秒表示。 <code>type</code> は <code>success</code> / <code>error</code>
<code>showConfirmModal()</code> / <code>closeConfirmModal()</code>	削除確認モーダルの開閉
<code>updateCharCount(inputId, countId)</code>	文字数カウンターの更新
<code>updateStat(totalCount)</code>	サイドバー下部のタスク数表示を更新

index.html

全画面のHTML構造を定義するエントリファイル。ボタンの `onclick` 属性から `main.js` の関数を直接呼び出す設計になっている。

ファイル	役割
#view-list	タスク一覧画面（デフォルト表示）
#view-add-task	タスク登録画面
#view-detail	タスク詳細画面
#view-edit	タスク編集画面
#view-add-label	ラベル管理画面
#confirm-modal	削除確認モーダル
#toast	トーストメッセージ枠

※ JSファイルの読み込み順序は `storage` → `task` → `label` → `filter` → `notification` → `ui` → `main` の順。依存関係があるため変更しないこと。

担当2:タスク更新・通知機能

js/task.js(更新・検索系)

タスクのCRUD操作とバリデーションを担当するモジュール。DOMには触れず、データ操作のみを行う。担当2が使う関数は更新・検索系。

関数	役割
<code>validate({ name, detail, deadline })</code>	入力値を検証してエラーメッセージの配列を返す。エラーがなければ空配列
<code>update(tasks, id, fields)</code>	指定 id のタスクを fields で上書きして localStorage に保存し、更新後の配列を返す
<code>changeStatus(tasks, id, status)</code>	ステータス (todo / doing / done) だけを変更。update() のラッパー
<code>findById(tasks, id)</code>	id でタスクを検索。存在しなければ null を返す

js/notification.js

Service Worker (sw.js) の登録・管理と、タスク変化時の通知スケジュール同期を担当。直接 OS 通知を出すのではなく、sw.js に指示を送る役割に徹する。

関数	役割
<code>start(getTasksFn)</code>	アプリ起動時に呼ぶ。SW 登録 → 通知許可取得 → 初回同期 → 1時間ごとの定期同期を開始
<code>syncOnTaskChange()</code>	タスク追加・更新後に main.js から呼ぶ。SW に最新タスク一覧を再送信
<code>cancelTask(taskId)</code>	タスク削除・完了時に main.js から呼ぶ。対象タスクの通知タイマーをキャンセル

SW との通信プロトコル (postMessage)

関数	役割
<code>SYNC_TASKS { type, tasks }</code>	タスク一覧を同期。スケジュールの追加・削除を自動判定
<code>CANCEL_TASK { type, taskId }</code>	特定タスクの全タイマーをキャンセル
<code>RESET_ALL { type }</code>	全タイマーをリセット

sw.js

ブラウザの裏側で動く Service Worker 本体。ブラウザのタブを閉じてでも OS ネイティブ通知が届くのはこのファイルの役割。notification.js からメッセージを受け取り、setTimeout

でタイマーを管理する。

通知の種類とタイミング

種類	発火タイミング	メッセージ例
24時間前警告	期限の24時間前	☒ 明日が期限です：〇〇
期限切れ通知	期限ちょうど（または即時）	☒ 期限になりました：〇〇
繰り返し通知	期限切れ後 1時間ごと（最大24回）	☒ まだ未完了です：〇〇

タイマー管理の仕組み

タイマーは `scheduledTimers` という Map で管理する。キーは `taskId_warning` / `taskId_expired` / `taskId_repeat_N` の形式。同じキーのタイマーが既に存在する場合は重複登録しない。OS 側の重複防止には `showNotification` の `tag` オプションを使用。

※ `sw.js` は `index.html` と同じフォルダに置くこと。サブフォルダに置くと Service Worker のスコープが狭くなり通知が届かなくなる。

担当3:タスク登録・削除

js/task.js(登録・削除系)

タスクのCRUD操作とバリデーションを担当するモジュール。DOMには触れず、データ操作のみを行う。担当3が使う関数は登録・削除系。

関数	役割
<code>validate({ name, detail, deadline })</code>	入力値を検証してエラーメッセージの配列を返す。エラーがなければ空配列。バリデーションルール：名前は1~30文字必須、詳細は100文字以下、期限は必須
<code>create(tasks, { name, detail, deadline, type, label })</code>	新規タスクオブジェクトを生成して配列の先頭に追加し <code>localStorage</code> に保存。自動付与フィールド： <code>id (Date.now) / status (todo) / createdAt / notified (false)</code>
<code>remove(tasks, id)</code>	指定 <code>id</code> のタスクを配列から削除して <code>localStorage</code> に保存。更新後の配列を返す

js/storage.js

`localStorage` への読み書きを一元管理するモジュール。他のファイルが直接 `localStorage` を触ることはなく、必ずこのファイルを経由する。キー名の変更や保存形式の変更はここだけ修正すればよい設計。

関数	役割
<code>loadTasks()</code>	タスク一覧を読み込む。パース失敗時は空配列を返す
<code>saveTasks(tasks)</code>	タスク一覧を JSON 文字列で保存
<code>loadLabels()</code>	ラベル一覧を読み込む。初回起動時はデフォルト5科目（数学・物理・英語・化学・情報）を返す
<code>saveLabels(labels)</code>	ラベル一覧を JSON 文字列で保存

ストレージキー

関数	役割
<code>'taskflow_tasks'</code>	タスク一覧の保存先キー
<code>'taskflow_labels'</code>	ラベル一覧の保存先キー

js/main.js(登録・削除に関わる部分)

フォームの入力値取得 → バリデーション → 保存 → 画面遷移 → 通知同期という一連の流れをつなぐエントリポイント。担当3が触る関数は以下の5つ。

関数	役割
registerTask()	登録フォームの値を ui.js から取得 → task.js でバリデーション → task.js で保存 → 通知同期 → 一覧画面へ遷移
cancelForm()	登録を中断してフォームをリセットし一覧画面へ戻る (☒ボタン・キャンセルボタン)
confirmDelete(id)	削除対象の id を保持して確認モーダルを表示
deleteTask()	通知キャンセル → task.js で削除 → モーダルを閉じる → 一覧画面へ遷移
closeModal()	削除確認モーダルを閉じるだけ (削除はしない)

タスクオブジェクトの構造

登録時に生成されるタスクオブジェクトのフィールド一覧：

フィールド	型	説明
id	string	Date.now() の文字列。一意なID
name	string	タスク名 (1~30文字)
detail	string	詳細 (0~100文字)
deadline	string	datetime-local 形式の文字列
type	string	”課題” または ”テスト”
label	string	選択したラベル名 (空文字もあり)
status	string	”todo” / ”doing” / ”done”
createdAt	string	ISO 8601 形式の登録日時
notified	boolean	期限切れ通知を送信済みかどうか

担当4:タスクのフィルタリング・ラベル登録

js/filter.js

ステータスフィルタとラベルフィルタの状態管理と絞り込みロジックを担当するモジュール。データもDOMも持たず、「どの条件で絞るか」の状態だけを管理する純粋なロジック層。

管理する状態

関数	役割
currentStatus	"all" / "todo" / "doing" / "done" のいずれか。初期値は "all"
selectedLabels	選択中のラベル名の配列。複数選択可能。初期値は空配列

公開関数

関数	役割
setStatus(status)	ステータスフィルタを設定。タブクリック時に main.js から呼ぶ
getStatus()	現在のステータスフィルタを返す
toggleLabel(label)	指定ラベルの選択・解除を切り替える
clearLabels()	ラベルフィルタをすべて解除
getSelectedLabels()	選択中ラベルの配列のコピーを返す
hasLabelFilter()	ラベルフィルタが1つ以上有効なら true を返す
apply(tasks)	現在の条件で tasks 配列を絞り込んで返す。元の配列は変更しない

js/label.js

ラベルのCRUD操作とバリデーションを担当するモジュール。DOMには触れず、データ操作のみを行う。

関数	役割
validate(name, existingLabels)	ラベル名を検証してエラーメッセージの配列を返す。バリデーションルール：空文字不可 / 30文字以下 / 同名ラベルの重複不可
add(labels, name)	ラベルを配列末尾に追加して localStorage に保存。更新後の配列を返す
remove(labels, name)	指定ラベルを削除して localStorage に保存。更新後の配列を返す

js/main.js(フィルタリング・ラベルに関わる部分)

担当4が触る関数はフィルタ操作とラベル管理の計6つ。

関数	役割
<code>filterStatus(status)</code>	ステータスタブクリック時に呼ばれる。 <code>filter.js</code> に状態をセット → タブのUIを更新 → 一覧を再描画
<code>toggleLabelFilter()</code>	ラベル絞り込みボタンクリック時。ドロップダウンの表示・非表示を切り替える
<code>_handleLabelFilterToggle(label, checked)</code>	チェックボックス変更時。 <code>filter.js</code> に <code>toggleLabel</code> → ボタンの強調表示更新 → 一覧再描画
<code>clearLabelFilter()</code>	絞り込み解除ボタン。フィルタをクリア → ドロップダウンを閉じる → 一覧再描画
<code>addLabel()</code>	ラベル追加ボタン。 <code>label.js</code> でバリデーション・保存 → 各UIへの反映
<code>_handleDeleteLabel(label)</code>	チップの✕ボタン。 <code>label.js</code> で削除 → 各UIへの反映

フィルタリングの処理フロー

ステップ	処理	担当ファイル
1	ユーザーがタブ/チェックボックスを操作	<code>index.html</code>
2	<code>main.js</code> のイベントハンドラが呼ばれる	<code>js/main.js</code>
3	<code>filter.js</code> の状態を更新	<code>js/filter.js</code>
4	<code>filter.apply(tasks)</code> で絞り込み実行	<code>js/filter.js</code>
5	<code>ui.renderTaskList()</code> で再描画	<code>js/ui.js</code>

※ ラベルを削除しても、そのラベルが設定済みのタスクのラベルフィールドは残る。一覧上では「ラベルなし」と同じ扱いになる。